

PORTADA

Técnicas de Programación

Unidad 0 — Vectores en Java

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



APERTURA

**Una variable para cada dato...
o un vector para todos ellos.**



CONCEPTO

Qué es un arreglo

Un **arreglo** (*array*) es una estructura de datos de **tamaño fijo** que guarda varios valores del **mismo tipo**, uno detrás de otro en memoria, accesibles por su posición (índice), que empieza en `0`.

```
int[] notas = new int[5];          // arreglo vacío de 5 enteros
int[] edades = {18, 21, 19, 25}; // arreglo inicializado directamente
```

El tamaño se fija al crear el arreglo y no puede cambiar durante la ejecución — si se necesita más espacio, hay que crear un arreglo nuevo.



CONCEPTO

Acceder y modificar posiciones

```
int[] edades = {18, 21, 19, 25};  
System.out.println(edades[0]); // 18 – primer elemento  
edades[1] = 22;                // modifica la segunda posición  
int ultimo = edades[edades.length - 1]; // último elemento
```

`edades.length` da el tamaño del arreglo (no es un método, es un atributo). Intentar acceder a una posición fuera de rango (`edades[10]` en un arreglo de 4) lanza `ArrayIndexOutOfBoundsException` .

CONCEPTO

Recorrer un arreglo

```
double[] notas = {3.5, 4.0, 2.8, 4.5};  
for (int i = 0; i < notas.length; i++) {  
    System.out.println("Posición " + i + ": " + notas[i]);  
}
```

Este es el patrón más común para recorrer arreglos cuando se necesita conocer el índice — por ejemplo, para modificar una posición específica mientras se recorre.

CONCEPTO

Matrices

Java permite declarar arreglos de varios subíndices. Con dos, se comportan como una **matriz** (filas y columnas); con tres, como un cubo — más allá de eso se pierde la intuición geométrica, pero la sintaxis sigue funcionando.

```
int[][] matriz = new int[3][3];    // matriz de 3x3, todo en 0
matriz[1][2] = 7;                  // fila 1, columna 2
int[][] tablero = {
    {1, 2, 3},
    {4, 5, 6}
};
```



CONCEPTO

Recorrer una matriz

```
for (int fila = 0; fila < matriz.length; fila++) {  
    for (int col = 0; col < matriz[fila].length; col++) {  
        System.out.print(matriz[fila][col] + " ");  
    }  
    System.out.println();  
}
```

Recorrer una matriz necesita **dos ciclos anidados**: el externo avanza por filas, el interno por columnas de esa fila.

CONCEPTO

ArrayList: arreglos con tamaño dinámico

	Array	ArrayList
Tamaño	Fijo, se define al crear	Dinámico, crece y decrece
Agregar/quitar elementos	No — hay que crear uno nuevo	Sí, con <code>.add()</code> y <code>.remove()</code>
Tipos que acepta	Primitivos y objetos	Solo objetos (<code>Integer</code> , <code>String</code> , ...)

```
ArrayList<String> nombres = new ArrayList<>();  
nombres.add("Ana");  
nombres.add("Luis");  
nombres.remove("Ana");
```



CONTRA EJEMPLO

¿Cuál usar?

Array — cuando el tamaño es fijo y conocido de antemano, y se necesita el mejor rendimiento posible (por ejemplo, una matriz de 3x3 para un juego).

ArrayList — cuando no se sabe cuántos elementos habrá, o se necesita agregar/quitar elementos mientras el programa corre (por ejemplo, una lista de usuarios conectados).

No es una regla absoluta — es una decisión de diseño que se justifica según el problema.

SÍNTESIS

Lo que hay que llevarse de hoy

1. Un arreglo guarda varios valores del mismo tipo, con tamaño fijo, accesibles por índice desde 0.
2. Recorrer un arreglo con `for` es el patrón más común cuando se necesita el índice.
3. Una matriz es un arreglo de arreglos — recorrerla necesita dos ciclos anidados.
4. `ArrayList` resuelve el problema del tamaño fijo, a cambio de solo aceptar objetos.



INSTRUCCIONES

Taller de hoy

Taller de arreglos y matrices — 2 horas, en equipo

- Resolver los ejercicios de arreglos y matrices que el docente entrega al inicio del taller.
- Entregable: archivo(s) `.java` con la solución, subidos por el equipo a la tarea correspondiente en Moodle.
- Cada integrante debe poder explicar cualquier parte del código entregado.

Plataforma de entrega: Moodle · **La entrega cierra al terminar las dos horas del taller.**



CIERRE

Cierre de la Unidad 0

Con esta sesión termina la Unidad 0 — repaso de Java, paradigma imperativo y vectores.

- La próxima unidad (Unidad 2 — Paradigma Objetual) arranca en la semana 3 con clases, herencia y sobrecarga.
- La Unidad 1 (Paradigmas) se evalúa aparte con el Caso de Estudio 1 — sin clase catedrática dedicada.

